CardDemo daily transaction posting: wiki pages and I/O contract

Wiki pages generated by the FlowX.AI legacy transformation agent from AWS CardDemo (open source, Apache-2.0). CONTRACT.md is the input/output contract for the build.

Pages included:

- `02-processes/process-transaction-posting.md`
- `10-acceptance-criteria/acceptance-criteria-transaction-posting.md`
- `01-data-model/project-data-model.md`
- `05-triggers/trigger-inventory.md`
- `CONTRACT.md`

`02-processes/process-transaction-posting.md`

Process: Transaction Posting (Batch Pipeline)

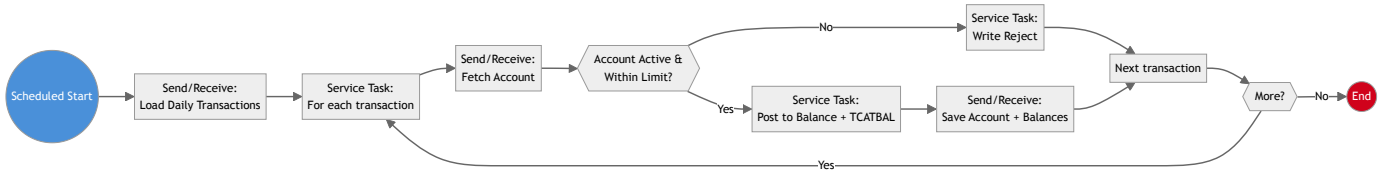
Powered by FLOWX.AI Coding Agents

Field	Value
Name	Transaction Posting
Type	Scheduled Process (orchestration)
Legacy Source	`CBTRN01C/CBTRN02C/CBTRN03C` via `POSTTRAN.jcl`, `COMBTRAN.jcl`, `DALYREJS.jcl`
Status	Draft
Related Triggers	Scheduled Trigger — nightly posting
Related Data	Transaction, Account, TransactionCategoryBalance

Business Description

Nightly pipeline that reads the day's incoming transactions, validates each against its account (status, credit limit), posts valid transactions to account balances and category balances (`TCATBALF`), and routes invalid ones to a rejects output. Replaces the `CBTRN*` batch posting chain driven by `POSTTRAN/COMBTRAN`, with rejects from `DALYREJS`.

Process Flow



Nodes

#	Node	Type	Description	Legacy Reference
1	Scheduled Start	Start Event (Scheduled Trigger)	Nightly batch cycle	`POSTTRAN.jcl` submission
2	Load Daily Transactions	Send/Receive Message Task	Read daily input (GDG generation)	`CBTRN01C` read
3	For each transaction	Service Task (loop)	Iterate input	main read loop
4	Fetch Account	Send/Receive Message Task	Read owning account	`READ ACCTFILE`
5	Account Active & Within Limit?	Exclusive Gateway	Validation	credit-limit & status checks
6	Post to Balance + TCATBAL	Service Task (NoOpHandler)	Update balances	`CBTRN02C` posting
7	Save Account + Balances	Send/Receive Message Task	Persist	`REWRITE ACCTFILE`, `TCATBALF`
8	Write Reject	Service Task	Route invalid	`DALYREJS.jcl`

Business Rules

Rule: PostingValidation

- **Legacy source:** `CBTRN02C` posting validation.
- **Logic:** account must be active; transaction must not exceed available credit (`creditLimit - currentBalance`); card active.
- **FLOWX implementation:** NoOpHandler business rule per transaction in the loop.

Rule: UpdateCategoryBalance

- **Logic:** add transaction amount to the matching `TransactionCategoryBalance` (key = account + type + category).

Data Mapping (collection loop)

- `inputKey`: `dailyTransactions` (source collection)
- `outputKey`: `postingResults` (posted + rejected)

Integration Touchpoints

Send Node	Topic	Payload	Receive	System
Load Daily Transactions	`ai.flowx.in.carddemo.datastore.loaddailytransactions`	`{ businessDate }`	Transaction[]	FlowX Persistence
Save Account + Balances	`ai.flowx.in.carddemo.datastore.postbatch`	`{ account, catBalances }`	ack	FlowX Persistence

Swimlanes

- **System** (unattended batch)

`{{TODO: Confirm GDG daily-cycle semantics — map to dated partitions/snapshots in FlowX Persistence. See CBTRN01-03C source for exact reject reasons.}}`

---

*Powered by **FLOWX.AI** Coding Agents*

Acceptance Criteria: Transaction Posting

> **Powered by FLOWX.AI Coding Agents** · `@regression`

Verifies Transaction Posting (`CBTRN01-03C`/POSTTRAN). **Critical regression** — balances must match the legacy batch exactly.

Configuration Dependencies

Config	Effect
Business date / cycle	Which daily file is posted

Scenarios

#	Scenario	Steps	Expected Result
1	Post a valid daily file (happy)	Run with a known daily file	All valid txns posted; balances match legacy
2	Over-limit transaction (gateway → reject)	Txn exceeds available credit	Routed to rejects, not posted
3	Inactive account (gateway → reject)	Account status N	Rejected
4	Category balance update (business rule)	Valid txn	TCATBAL for account+type+category increased
5	Empty daily file (edge)	No input	Run completes, nothing posted
6	Mixed valid/invalid	File with both	Valid posted, invalid rejected; counts match legacy `DALYREJS`

Data Mapping Verification

Flow	Verify
Loop (`inputKey=dailyTransactions`)	each transaction processed once
Save Account + Balances	account balance + TCATBAL persisted

Legacy Comparison

Input	Legacy `CBTRN02C`	FLOWX
daily file	posted balances + rejects	identical balances + reject set

---

Project Data Model — AWS CardDemo → FLOWX.AI

> **Powered by FLOWX.AI Coding Agents**

This section documents all **Project Data Types** derived from the CardDemo copybooks (`/app/cpy`). Each VSAM record layout maps to a Project Data Type; COBOL `05`-level items become attributes. Packed/zoned/signed numerics (`COMP-3`, `S9(n)V99`) map to `NUMBER`, fixed-width `X(10)` dates map to `DATE`, and FILLER tails are dropped. Enumerated reference codes are documented in Enumerations.

> **Type-conversion note:** `S9(10)V99` → `NUMBER` (decimal, scale 2); `9(n)` → `NUMBER` (integer); `X(n)` → `STRING` (maxLength n); `X(10)` date → `DATE`; `X(26)` timestamp → `DATE` (timestamp). The `ACCT-EXPIRAION-DATE` legacy typo is corrected to `expirationDate` — see `{{DECISION}}` below.

Business Categories

Category	Types
Core Processing	Account, Transaction, TransactionCategoryBalance
Domain	Customer, Card, CardXref
Reference / Config	TransactionType, TransactionCategory
Security	SecurityUser
Navigation (internal)	SessionContext

Account

Replaces the ACCOUNT-RECORD VSAM KSDS (`ACCTFILE` / `CVACT01Y`, 300 bytes). Key: `accountId`.

Attribute	Type	Required	Constraints	Description	Legacy Source
accountId	NUMBER	Yes	11 digits, unique key	Account identifier	`ACCT-ID 9(11)`
activeStatus	ENUM	Yes	AccountStatus	Active status flag	`ACCT-ACTIVE-STATUS X(1)`
currentBalance	NUMBER	Yes	decimal(12,2), signed	Current account balance	`ACCT-CURR-BAL S9(10)V99`
creditLimit	NUMBER	Yes	decimal(12,2), signed	Credit limit	`ACCT-CREDIT-LIMIT S9(10)V99`
cashCreditLimit	NUMBER	Yes	decimal(12,2), signed	Cash credit limit	`ACCT-CASH-CREDIT-LIMIT S9(10)V99`
openDate	DATE	Yes		Account open date	`ACCT-OPEN-DATE X(10)`
expirationDate	DATE	Yes		Account expiration date	`ACCT-EXPIRAION-DATE X(10)` *(legacy typo)*
reissueDate	DATE	No		Account reissue date	`ACCT-REISSUE-DATE X(10)`
currentCycleCredit	NUMBER	Yes	decimal(12,2), signed	Current-cycle credits	`ACCT-CURR-CYC-CREDIT S9(10)V99`

Attribute	Type	Required	Constraints	Description	Legacy Source
currentCycleDebit	NUMBER	Yes	decimal(12,2), signed	Current-cycle debits	`ACCT-CURR-CYC-DEBIT S9(10)V99`
addressZip	STRING	No	max 10	Account ZIP code	`ACCT-ADDR-ZIP X(10)`
groupId	STRING	No	max 10	Disclosure/interest group id	`ACCT-GROUP-ID X(10)`

`{{DECISION: Correct the legacy field name typo ACCT-EXPIRAION-DATE → expirationDate (recommended), or preserve the misspelling for round-trip fidelity? ETL must map the legacy column either way.}}`

Used by: Account View, Account Update, Bill Payment, Transaction Posting, Interest Calculation.

Customer

Replaces CUSTOMER-RECORD (`CUSTFILE` / `CVCUS01Y`, 500 bytes). Key: `customerId`.

Attribute	Type	Required	Constraints	Description	Legacy Source
customerId	NUMBER	Yes	9 digits, unique key	Customer identifier	`CUST-ID 9(9)`
firstName	STRING	Yes	max 25	First name	`CUST-FIRST-NAME X(25)`
middleName	STRING	No	max 25	Middle name	`CUST-MIDDLE-NAME X(25)`
lastName	STRING	Yes	max 25	Last name	`CUST-LAST-NAME X(25)`
addressLine1	STRING	Yes	max 50	Address line 1	`CUST-ADDR-LINE-1 X(50)`
addressLine2	STRING	No	max 50	Address line 2	`CUST-ADDR-LINE-2 X(50)`
addressLine3	STRING	No	max 50	Address line 3	`CUST-ADDR-LINE-3 X(50)`
addressStateCode	STRING	No	max 2	State code	`CUST-ADDR-STATE-CD X(2)`
addressCountryCode	STRING	No	max 3	Country code	`CUST-ADDR-COUNTRY-CD X(3)`
addressZip	STRING	No	max 10	ZIP code	`CUST-ADDR-ZIP X(10)`
phoneNumber1	STRING	No	max 15	Primary phone	`CUST-PHONE-NUM-1 X(15)`
phoneNumber2	STRING	No	max 15	Secondary phone	`CUST-PHONE-NUM-2 X(15)`
ssn	NUMBER	Yes	9 digits (PII)	Social security number	`CUST-SSN 9(9)`
govtIssuedId	STRING	No	max 20	Government-issued id	`CUST-GOVT-ISSUED-ID X(20)`
dateOfBirth	DATE	No		Date of birth	`CUST-DOB-YYYY-MM-DD X(10)`
eftAccountId	STRING	No	max 10	EFT account id	`CUST-EFT-ACCOUNT-ID X(10)`
primaryCardHolderInd	STRING	No	max 1	Primary-holder indicator	`CUST-PRI-CARD-HOLDER-IND X(1)`
ficoCreditScore	NUMBER	No	300–850	FICO credit score	`CUST-FICO-CREDIT-SCORE 9(3)`

`{{TODO: SSN and govt id are PII — define masking/encryption and access restrictions in FLOWX.}}`

Used by: Account View, Account Update, Authorization.

Card

Replaces CARD-RECORD (`CARDFILE` + `CARDAIX` / `CVACT02Y`, 150 bytes). Key: `cardNumber`; alternate index on `cardAccountId`.

Attribute	Type	Required	Constraints	Description	Legacy Source
cardNumber	STRING	Yes	16 chars, unique key (PCI)	Card number	`CARD-NUM X(16)`
cardAccountId	NUMBER	Yes	11 digits, AIX	Owning account id	`CARD-ACCT-ID 9(11)`
cvvCode	NUMBER	Yes	3 digits (PCI)	Card verification value	`CARD-CVV-CD 9(3)`
embossedName	STRING	Yes	max 50	Name embossed on card	`CARD-EMBOSSED-NAME X(50)`
expirationDate	DATE	Yes		Card expiration date	`CARD-EXPIRAION-DATE X(10)` *(legacy typo)*
activeStatus	ENUM	Yes	CardStatus	Card active status	`CARD-ACTIVE-STATUS X(1)`

`{{TODO: cardNumber and cvvCode are PCI-sensitive — define tokenization/masking strategy.}}`

Used by: Card List, Card View, Card Update, Card Security, Authorization.

CardXref

Replaces CARD-XREF-RECORD (`XREFFILE` + `CXACAIX` / `CVACT03Y`, 50 bytes). Links card ↔ customer ↔ account; alternate index on `accountId`.

Attribute	Type	Required	Constraints	Description	Legacy Source
cardNumber	STRING	Yes	16 chars, key	Card number	`XREF-CARD-NUM X(16)`
customerId	NUMBER	Yes	9 digits	Customer id	`XREF-CUST-ID 9(9)`
accountId	NUMBER	Yes	11 digits, AIX	Account id	`XREF-ACCT-ID 9(11)`

Used by: Account View, Card List, Transaction Add, Authorization.

Transaction

Replaces TRAN-RECORD (`TRANFILE` / `CVTRA05Y`, 350 bytes). Key: `transactionId`.

Attribute	Type	Required	Constraints	Description	Legacy Source
transactionId	STRING	Yes	16 chars, unique key	Transaction identifier	`TRAN-ID X(16)`
typeCode	ENUM	Yes	TransactionType	Transaction type code	`TRAN-TYPE-CD X(2)`
categoryCode	ENUM	Yes	TransactionCategory	Transaction category code	`TRAN-CAT-CD 9(4)`
source	STRING	No	max 10	Transaction source	`TRAN-SOURCE X(10)`
description	STRING	No	max 100	Transaction description	`TRAN-DESC X(100)`
amount	NUMBER	Yes	decimal(11,2), signed	Transaction amount	`TRAN-AMT S9(9)V99`
merchantId	NUMBER	No	9 digits	Merchant id	`TRAN-MERCHANT-ID 9(9)`
merchantName	STRING	No	max 50	Merchant name	`TRAN-MERCHANT-NAME X(50)`
merchantCity	STRING	No	max 50	Merchant city	`TRAN-MERCHANT-CITY X(50)`
merchantZip	STRING	No	max 10	Merchant ZIP	`TRAN-MERCHANT-ZIP X(10)`

Attribute	Type	Required	Constraints	Description	Legacy Source
cardNumber	STRING	Yes	16 chars	Card used	`TRAN-CARD-NUM X(16)`
originTimestamp	DATE	No	timestamp	Origination timestamp	`TRAN-ORIG-TS X(26)`
processTimestamp	DATE	No	timestamp	Processing timestamp	`TRAN-PROC-TS X(26)`

Used by: Transaction List, Transaction View, Transaction Add, Transaction Posting, Reports.

TransactionCategoryBalance

Replaces TRAN-CAT-BAL-RECORD (`TCATBALF` / `CVTRA01Y`, 50 bytes). Composite key: `accountId` + `typeCode` + `categoryCode`.

Attribute	Type	Required	Constraints	Description	Legacy Source
accountId	NUMBER	Yes	11 digits, key part	Account id	`TRANCAT-ACCT-ID 9(11)`
typeCode	ENUM	Yes	TransactionType, key part	Transaction type	`TRANCAT-TYPE-CD X(2)`
categoryCode	ENUM	Yes	TransactionCategory, key part	Category	`TRANCAT-CD 9(4)`
balance	NUMBER	Yes	decimal(11,2), signed	Category running balance	`TRAN-CAT-BAL S9(9)V99`

Used by: Transaction Posting, Interest Calculation, Reports.

SecurityUser

Replaces SEC-USER-DATA (`USRSEC` / `CSUSR01Y`). Key: `userId`. Target FLOWX model gates access with roles; plaintext password is replaced by IdP credentials.

Attribute	Type	Required	Constraints	Description	Legacy Source
userId	STRING	Yes	8 chars, unique key	User id	`SEC-USR-ID X(8)`
firstName	STRING	Yes	max 20	First name	`SEC-USR-FNAME X(20)`
lastName	STRING	Yes	max 20	Last name	`SEC-USR-LNAME X(20)`
password	STRING	—	**DO NOT MIGRATE**	Plaintext password (legacy debt)	`SEC-USR-PWD X(8)`
userType	ENUM	Yes	UserType	Admin (`A`) / User (`U`)	`SEC-USR-TYPE X(1)`

`{{DECISION: Do NOT migrate the plaintext password column. Provision users in a modern IdP and map userType → FLOWX roles. Confirm IdP (Keycloak/other).}}`

Used by: Signon, User Administration.

SessionContext

Internal process data replacing the navigation portion of `CARDDEMO-COMMAREA` (`COCOM01Y`). Not persisted — lives in process instance data. Documented here for traceability; in FLOWX most of these become implicit process state.

Attribute	Type	Description	Legacy Source
userId	STRING	Signed-on user id	`CDEMO-USER-ID X(8)`
userType	ENUM	UserType	`CDEMO-USER-TYPE X(1)` (88 A/U)
selectedCustomerId	NUMBER	Current customer context	`CDEMO-CUST-ID 9(9)`
selectedAccountId	NUMBER	Current account context	`CDEMO-ACCT-ID 9(11)`
selectedCardNumber	STRING	Current card context	`CDEMO-CARD-NUM 9(16)`

`{{DECISION: The from/to tranid/program and last-map/mapset fields (CDEMO-FROM-*, CDEMO-TO-*, CDEMO-LAST-MAP) are pure navigation plumbing — replace with FLOWX process transitions and do not model them as data. Confirm.}}`

See Data Model Diagram for relationships.

---

*Powered by **FLOWX.AI** Coding Agents*

Trigger Inventory — AWS CardDemo → FLOWX.AI

Powered by FLOWX.AI Coding Agents

How each process starts. Legacy online transactions are user-initiated (CICS tran codes / menu); batch is JCL job submission; authorization is MQ-message driven.

Manual / Start-Event Triggers (Online)

Online transactions become FLOWX processes started from the FLOWX SDK UI or navigation. Each legacy tran code maps to a manual start.

Process	Legacy Entry	FLOWX Trigger
Signon	CC00	Manual (login page)
Main Menu Router	CM00	Call Activity from Signon / navigation
Admin Menu Router	CA00	Call Activity from Signon / navigation
Account / Card / Transaction / Bill / Reports	CAVW…CR00	Manual from menu navigation
User Administration	CU00–CU03	Manual from admin menu

Scheduled Triggers (Batch JCL)

The 37 JCL jobs become FLOWX **Scheduled Triggers** starting orchestration processes. Key schedules (cron to be defined per operations window):

Scheduled Process	Legacy JCL	Suggested Schedule	Notes
Transaction Posting	`POSTTRAN`, `COMBTRAN`, `DALYREJS`	Nightly	GDG daily cycle → dated partition
Interest Calculation	`INTCALC`	Monthly cycle	Depends on disclosure groups
Statement Generation	`CREASTMT`, `TXT2PDF1`	Monthly	After posting + interest
Data Export	`CBEXPORT`, `FTPJCL`	Daily/on-demand	Export file delivery
Data Import	`CBIMPORT`	On-demand	Bulk load
Purge Expired Authorizations	`CBPAUP0C`/`CBPAUP0J`	Daily	Auth module
Reference/report jobs	`TRANREPT`, `PRTCATBL`	Scheduled/on-demand	Reporting

Replaced infrastructure: the Assembler/COBOL timers (`MVSWAIT`/`COBSWAIT`, `WAITSTEP.jcl`, `INTRDRJ1/2`) become platform scheduler waits — no explicit timer program needed.

File-definition / utility JCL (`ACCTFILE`, `CARDFILE`, `CUSTFILE`, `XREFFILE`, `TRANFILE`, `TCATBALF`, `DEFGDGB/D`, `OPENFIL`, `CLOSEFIL`, `ESDSRRDS`, `DEFCUST`, `DISCGRP`, `DUSRSECJ`, `READACCT/CARD/CUST/XREF`): these define/load/seed datasets. In FLOWX they become **one-time data-migration ETL** (see implementation-overview), not runtime triggers.

Message Triggers

Process	Legacy Source	FLOWX Trigger
Authorization	`COPAUA0C` MQ-triggered (CP00)	**Message Trigger** — Authorization Gateway publishes requests to a Kafka topic that message-starts the process

Email Triggers

None in the legacy system. `{{TODO: Evaluate whether inbound email (e.g., statement requests, dispute intake) should start any process — a new capability.}}`

---

*Powered by **FLOWX.AI** Coding Agents*

Candidate I/O contract

What a migrated implementation must produce so the scorer can compare it with the oracle. The contract fixes **formats only**. It says nothing about behaviour, so it may be handed to a builder without leaking the answers.

Inputs (given per run)

The same files the COBOL chain reads, one record per line, fixed width, layouts as in the copybooks:

File	Copybook	Record
`dailytran.dat`	CVTRA06Y	350
`acctdata.dat`	CVACT01Y	300
`cardxref.dat`	CVACT03Y	50
`tcatbal.dat`	CVTRA01Y	50
`discgrp.dat`	CVTRA02Y	50
`custdata.dat`, `carddata.dat`, `trantype.dat`, `trancatg.dat`	CUSTREC, CVACT02Y, CVTRA03Y, CVTRA04Y	500, 150, 60, 60

Run parameters: business date `2022-07-18`, INTCALC parameter `2022071800`.

Step isolation. A candidate for INTCALC receives the oracle's POSTTRAN outputs as its account and category-balance inputs, so an INTCALC result is never penalised for a POSTTRAN error.

Outputs

One JSON file per output: a list of objects, in any order. Field names and types are fixed. Strings may omit trailing spaces. Amounts are JSON strings or numbers holding at most two decimals (`"14.56"`, `-500`).

File	Step	Fields
`posttran_transact.json`	POSTTRAN	transaction fields (below) for every posted transaction
`posttran_dalyrejs.json`	POSTTRAN	transaction fields + `reason` (int) + `reason_desc` for every rejected transaction
`posttran_acctdata.json`	POSTTRAN	account fields for every account, after posting
`posttran_tcatbal.json`	POSTTRAN	`acct_id, type_cd, cat_cd, bal` for every category balance, after posting
`intcalc_systran.json`	INTCALC	transaction fields for every interest transaction written
`intcalc_acctdata.json`	INTCALC	account fields for every account, after interest

- Transaction fields: `tran_id, type_cd, cat_cd (int), source, desc, amt, merchant_id (int), merchant_name, merchant_city, merchant_zip, card_num, orig_ts, proc_ts`
- Account fields: `acct_id (int), active_status, curr_bal, credit_limit, cash_credit_limit, open_date, expiration_date, reissue_date, curr_cyc_credit, curr_cyc_debit, addr_zip, group_id`

A run may instead leave the oracle's fixed-width dumps (`<name>.txt`). The scorer reads either form.

Optional `flags.json`: the uncertainties the pipeline declared **before** scoring, as `[{"rule": "R-INT-TRUNC", "source": "<artefact path and line>"}]` or `[{"output": "...", "key": "...", "source": "..."}]`. See

PROTOCOL.md §6 for what counts as a flag.